Technical Guide for ArcGUI Road Segmentation

October 2026

Contents

1 GitHub Repository and Deployment Files **5**

2 Guide Scope and Current Version **6**

3 General Function **6**
- 3.1 Recommended and Additional Model Variants . . . 7
- 3.2 Fixed Model . . . 8
- 3.3 Fixed Imagery and Preprocessing Configuration . . . 8
- 3.4 Probability Reconstruction . . . 9

4 Environment Setup **9**
- 4.1 Clone the ArcGIS Python Environment . . . 9
- 4.2 GPU / CUDA Environment . . . 10
- 4.3 CPU-Only Environment . . . 10
- 4.4 ArcGIS Image Analyst Requirement . . . 11
- 4.5 Verify ONNX Runtime and a Model with `check.py` . . . 11
- 4.6 Simple Standalone Folder Inference Example . . . 11

5 ArcGIS Pro Tool Installation and Main GUI **12**
- 5.1 Add the Toolboxes . . . 12
- 5.2 Main Road Segmentation Interface . . . 13
- 5.3 Recommended Starting Batch Sizes . . . 15

6 Coordinate Systems, Extents, and Geometry Handling **15**
- 6.1 ArcGIS Pro Environment Settings . . . 15
- 6.2 Map / Export Coordinate System . . . 17
- 6.3 Extent Layers with a Defined CRS . . . 17
- 6.4 Extent Layers with an Unknown CRS . . . 17
- 6.5 Polygon Validation, Selections, and Definition Queries . . . 18
- 6.6 Single Layer, Multiple Polygons, and Multiple Layer Entries . . . 18

7 Fixed Model Windows and Overlap **18**

8 Processing Mode 1: Direct ArcGIS 1024-Tile Export **19**

9 Processing Mode 2: Optimized Large-Area Export **20**
- 9.1 Default Large-Block Limit . . . 20
- 9.2 Balanced Non-Overlapping Partitioning . . . 20
- 9.3 Skip Large Blocks That Do Not Intersect the Target . . . 21
- 9.4 Irregular Polygon Extents . . . 22
- 9.5 Image-Service Request Limits and Automatic Smaller-Block Retry . . . 23

9.6 Multiple TIFFs from One Large Export . . . 23
9.7 Large-Block Boundary Limitation . . . 23

10 Reusing Previously Exported Imagery **24**

11 Debug Footprint Preview **24**

12 Georeference and Raster-Alignment Validation **25**

13 Probability Products, Thresholding, and Final Outputs **26**
13.1 Probability Mosaic . . . 26
13.2 Finalization Products . . . 26
13.3 Representative Output Structure . . . 27

14 Probability Map Threshold Toolbox **28**
14.1 Tool 1: Load / Refresh Threshold Preview . . . 28
14.2 Tool 2: Generate Binary Road Mask . . . 29
14.3 Native ArcGIS Symbology as a Slider-Like Alternative . . . 29
14.4 Recommended Threshold-Selection Workflow . . . 29

15 Runtime Safeguards and Failure behaviour **30**
15.1 Output Directory and Disk Space . . . 30
15.2 Reusable Cache Validation . . . 30
15.3 Black or Missing Imagery . . . 30
15.4 Inference Failures . . . 31
15.5 All Extents or All Debug Jobs Failing . . . 31

16 Troubleshooting **31**
16.1 CUDA Model Selected but CUDA Is Not Available . . . 31
16.2 CPU Inference Is Very Slow . . . 31
16.3 GPU Out of Memory . . . 31
16.4 Model Filename Is Rejected . . . 31
16.5 Input Raster Layer Cannot Be Found . . . 32
16.6 The Map Uses Degrees or Feet . . . 32
16.7 Extent Layer Has an Unknown CRS . . . 32
16.8 Debug Preview Reports NaN Extent Values . . . 32
16.9 Optimized Large Export Is Rejected by the Image Service . . . 32
16.10 A Large-Block Folder Contains a Correct TIFF and a Black TIFF . . . 33
16.11 Every TIFF for a Retained Block Is Black . . . 33
16.12 Large Blocks Appear in the Wrong Location . . . 33
16.13 Irregular Polygon Produces Rectangular Temporary Imagery . . . 33
16.14 Prediction Artifacts Near Model-Tile Edges . . . 33
16.15 Prediction Artifacts at Large-Block Boundaries . . . 33

16.16Debug Preview Is Too Dense 34
16.17A Cache Exists but ArcGUI Exports Imagery Again 34
16.18The GUI Shows a CACHE HIT Warning 34
16.19Threshold Preview Does Not Update Continuously While Dragging 34
16.20All Extents Failed but ArcGIS Previously Said “Succeeded” 34

17 Recommended Operational Workflow **35**

18 Deployment Summary **35**

1 GitHub Repository and Deployment Files

The project repository is available at:

`https://github.com/majrblais/ArcGUI`

The ONNX model files are distributed separately because of their size.

The current deployment package can contain the following model files:

```
segformer_cuda_fp16.onnx
best_unet_resnet101_norm_1024_cuda_fp16.onnx
segformer_last_cuda_fp16.onnx
last_unet_resnet101_norm_1024_cuda_fp16.onnx
segformer_cpu_fp32_optimized.onnx
old_milestone_unet_resnet101_norm_1024.onnx
```

For normal production use, the two recommended models are `segformer_cuda_fp16.onnx` and `best_unet_resnet101_norm_1024_cuda_fp16.onnx`. The SegFormer model is the primary final model and provides the strongest overall segmentation quality in the final evaluation. The best U-Net ResNet101 CUDA model remains a very useful alternative because it produces broadly comparable road maps while requiring less inference time. The remaining CPU, last-checkpoint, and old-milestone models are included mainly for compatibility, comparison, reproducibility, or machines without CUDA rather than as the preferred production choices. All current models use the same RGB input size and preprocessing, so the practical model choice is mainly a balance between available hardware, inference speed, and the small quality differences observed during validation.

A practical deployment layout is:

```
ArcGUI/
|-- README.md
|-- requirements.txt
|-- check.py
|-- python_inference_script.py
|-- models/
|   |-- segformer_cuda_fp16.onnx
|   |-- best_unet_resnet101_norm_1024_cuda_fp16.onnx
|   |-- segformer_last_cuda_fp16.onnx
|   |-- last_unet_resnet101_norm_1024_cuda_fp16.onnx
|   |-- segformer_cpu_fp32_optimized.onnx
|   `-- old_milestone_unet_resnet101_norm_1024.onnx
`-- toolbox_files/
    |-- roadsegmentation_tool.pyt
    |-- roadsegmentation_python.py
    `-- RoadProbabilityThreshold.pyt
```

The primary road-segmentation toolbox consists of roadsegmentation_tool.pyt and roadsegmentation_python.py. These two files must remain together in the same folder because the Python toolbox explicitly loads the backend located beside the `.pyt` file. The optional RoadProbabilityThreshold.pyt toolbox is a separate post-processing utility that operates on a probability raster after inference.

2 Guide Scope and Current Version

This document describes the current hardened deployment workflow implemented by **ArcGUI Road Segmentation** . The codebase identifies itself internally as `5.4 FINAL r7` , while the ArcGIS Pro toolbox and tool are intentionally displayed simply as *ArcGUI Road Segmentation*.

The guide focuses on practical installation and operation: environment setup, recommended model selection, the main ArcGIS interface, direct and optimized imagery export, overlap, reusable imagery caching, probability outputs, threshold selection, and common failure messages. The complete technical report containing model training, dataset preparation, validation statistics, and model comparisons is separate from this deployment guide.

The neural network always receives fixed 1024×1024 RGB windows at 2 m/pixel. The user mainly chooses an extent, a model, an output folder, the overlap percentage, the processing mode, and the prediction threshold. The deployment also includes reusable exported-imagery caching so the same area and export configuration can be processed with another model without downloading the same imagery again. The current build performs extensive validation before and during processing so invalid coordinate systems, missing imagery, incompatible model files, damaged caches, incomplete exports, or partial predictions fail visibly instead of silently producing a misleading road map.

3 General Function

The trained road-segmentation models accept RGB imagery and return a single-channel road logit map. ArcGUI converts logits to probabilities with a sigmoid function, georeferences the probability outputs, averages overlapping model-window predictions, and applies the user-selected threshold only after the probability surface has been reconstructed. This separation between inference and thresholding allows the probability output to be inspected and re-thresholded later without rerunning the neural network.

3.1 Recommended and Additional Model Variants

Table 1: Current deployment model options.

Model	Target	Recommended use
SegFormer MiT-B5	CUDA required	**Primary recommended model.** Final SegFormer deployment model with the strongest overall final evaluation.
U-Net ResNet101	CUDA required	**Recommended faster alternative.** Broadly comparable road extraction performance with lower inference cost.
SegFormer MiT-B5 (last checkpoint)	CUDA required	Last-training-checkpoint variant retained mainly for comparison and reproducibility.
U-Net ResNet101 (last checkpoint)	CUDA required	Last-training-checkpoint variant retained mainly for comparison and reproducibility.
SegFormer MiT-B5 (CPU)	CPU only	Compatibility option for systems without a usable NVIDIA CUDA environment; substantially slower for large processing jobs.
Older U-Net ResNet101	CUDA preferred; CPU fallback	Historical milestone model retained for comparison, reproducibility, and backward compatibility.

The final evaluation showed that the trained models produced broadly similar road-segmentation results, so the normal operational choice is not based on a dramatic accuracy difference. For most users, `segformer_cuda_fp16.onnx` should be selected when the best final model is desired, while `best_unet_resnet101_norm_1024_cuda_fp16.onnx` is a strong choice when faster inference is important.

ArcGUI recognizes the model family from the filename rather than from one exact hard-coded filename. A filename containing `segformer` is treated as SegFormer MiT-B5, while a filename containing `unet` together with `resnet101` or `r101` is treated as U-Net ResNet101. A filename containing `cpu` is forced to CPU, a filename containing `cuda` or `fp16` requires CUDA, and a recognized filename containing neither uses CUDA when available with CPU fallback. The filenames distributed with the project should therefore be retained unless there is a specific reason to rename them.

3.2 Fixed Model

ArcGUI validates the selected ONNX model before inference. The deployment expects one RGB image input and one single-channel segmentation output. The image input must be a floating-point NCHW tensor, which is created by the GUI, with three channels and a spatial size of 1024×1024 when those dimensions are static. Dynamic batch models and static batch-1 models are supported. If a selected graph has a static batch size of one, ArcGUI automatically reduces a larger requested batch size to one. The model output must be floating point and compatible with a single road-logit channel. ArcGUI applies sigmoid to the logits before overlap averaging and rejects NaN or infinite model outputs.

3.3 Fixed Imagery and Preprocessing Configuration

Table 2: Fixed model and imagery assumptions.

Parameter	**Value**
Input imagery	RGB
Model input	1024×1024 pixels
Deployment cell size	2.0 metres per pixel
Input tensor type	FP32
Normalization	ImageNet
Model output	Single-channel road logits
Probability conversion	Sigmoid
Default main-tool threshold	0.50
Allowed overlap range	0–95%
Default overlap	50%
Extent policy	Exact selected polygon

The RGB image is scaled to the $[0, 1]$ range and normalized with:

$$\mu = [0.485,\ 0.456,\ 0.406], \qquad \sigma = [0.229,\ 0.224,\ 0.225].$$

The fixed ground resolution is **2.0 metres per pixel**. For this reason, production and debug planning require an active map with a projected coordinate system whose units are metres. A geographic coordinate system measured in degrees or a projected coordinate system measured in feet is rejected because a numeric cell size of 2.0 would represent the wrong ground resolution.

3.4 Probability Reconstruction

For every model window, ArcGUI obtains a road-probability array after sigmoid conversion. When several model windows cover the same pixel, ArcGUI averages their probability values before thresholding.

For a pixel (x, y) covered by $n(x, y)$ model windows:

$$P_{\text{final}}(x, y) = \frac{\sum_{i=1}^{n(x,y)} P_i(x, y)}{n(x, y)}.$$

The binary decision for threshold T is:

$$R(x, y) = \begin{cases} 1, & P_{\text{final}}(x, y) > T, \\ 0, & P_{\text{final}}(x, y) \leq T. \end{cases}$$

The strict comparison is therefore `probability > threshold`.

4 Environment Setup

The ArcGIS deployment requires ArcGIS Pro, a cloned ArcGIS Python environment, the ArcGIS Image Analyst extension, ONNX Runtime, OpenCV, NumPy, Pillow, and the remaining packages required by the repository. A CUDA-capable NVIDIA GPU is strongly recommended for routine large-area processing. The CPU SegFormer graph remains available as a compatibility option when CUDA is not available.

4.1 Clone the ArcGIS Python Environment

In ArcGIS Pro, open the Package Manager and clone the default ArcGIS Pro environment before installing additional packages.

A practical cloned-environment name is:

`torch_roads`

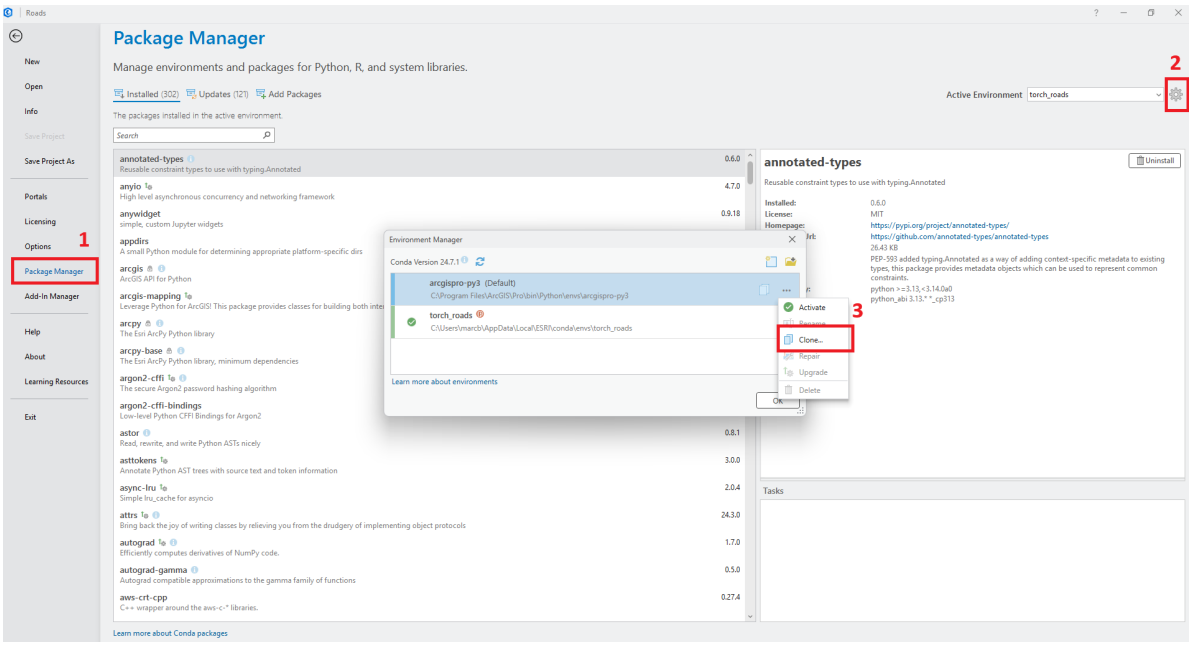


Figure 1: Cloning the ArcGIS Python environment using the Package Manager.

Activate the environment from an Anaconda Prompt:

```
conda activate torch_roads
```

Then install the repository dependencies appropriate for the workstation based on the GitHub instructions.

4.2 GPU / CUDA Environment

For a workstation with a compatible NVIDIA GPU, use one of the two recommended production models:

```
segformer_cuda_fp16.onnx
best_unet_resnet101_norm_1024_cuda_fp16.onnx
```

Use SegFormer when the primary final model is preferred. Use the best U-Net ResNet101 CUDA model when lower inference time is more important and a broadly comparable road map is sufficient. A correct GPU environment must expose `CUDAExecutionProvider` through ONNX Runtime. If a CUDA/FP16 model is selected and `CUDAExecutionProvider` cannot be activated, ArcGUI stops rather than silently running the model on CPU.

4.3 CPU-Only Environment

For a CPU-only workstation, the supplied compatibility model is:

```
segformer_cpu_fp32_optimized.onnx
```

This graph deliberately uses `CPUExecutionProvider` only. CPU inference is substantially

slower than CUDA inference, so the CPU graph is best treated as a fallback or compatibility option rather than the preferred large-area deployment path.

4.4 ArcGIS Image Analyst Requirement

Production imagery export uses *Export Training Data for Deep Learning* and therefore ArcGUI checks out the Image Analyst extension before processing. The current workflow does not require a separate Spatial Analyst checkout for its normal production path. If Image Analyst is unavailable, the production job stops with a licensing error before starting a long export.

4.5 Verify ONNX Runtime and a Model with `check.py`

The supplied `check.py` utility is intended as a quick environment and model test before using ArcGIS Pro. Launch `check.py` directly from the intended Python environment. The script opens a file-selection window and asks the user to select an ONNX model. It automatically detects SegFormer versus U-Net ResNet101, selects the expected CPU/CUDA execution mode from the filename, loads the ONNX graph, checks the fixed 1024×1024 RGB model contract, and runs an actual dummy inference.

For a CUDA model, a successful test ends with a message similar to:

```
SUCCESS - MODEL LOADED AND INFERENCE RAN WITH CUDA
```

For a CPU model, the success message reports CPU execution instead. This check is particularly useful after moving the project to another workstation or after changing the Python, ONNX Runtime, CUDA, or NVIDIA-driver environment.

4.6 Simple Standalone Folder Inference Example

The supplied `python_inference_script.py` is a deliberately simple example that demonstrates model inference without ArcGIS. Launch the script, select an ONNX model, and then select a folder containing images. The script accepts common image formats such as PNG, JPEG, TIFF, and BMP, resizes each image to the required model input, applies the same ImageNet normalization, runs one image at a time, and writes a binary road mask for every input image.

The output folder is created beside the input images using the selected model name, for example:

```
images/
|-- image_001.tif
|-- image_002.tif
‘-- masks_segformer_cuda_fp16/
    |-- image_001_mask.png
    ‘-- image_002_mask.png
```

The standalone script is a quick demonstration and model sanity check; it is **not** a replacement for the ArcGIS workflow because it does not perform georeferenced large-area export, overlap reconstruction, extent clipping, or GIS output management.

5 ArcGIS Pro Tool Installation and Main GUI

5.1 Add the Toolboxes

Open the ArcGIS Pro Catalog pane, add roadsegmentation_tool.pyt, and keep roadsegmentation_python.py in the same directory. Add RoadProbabilityThreshold.pyt as a second toolbox when probability-threshold exploration and permanent binary-mask generation are required.

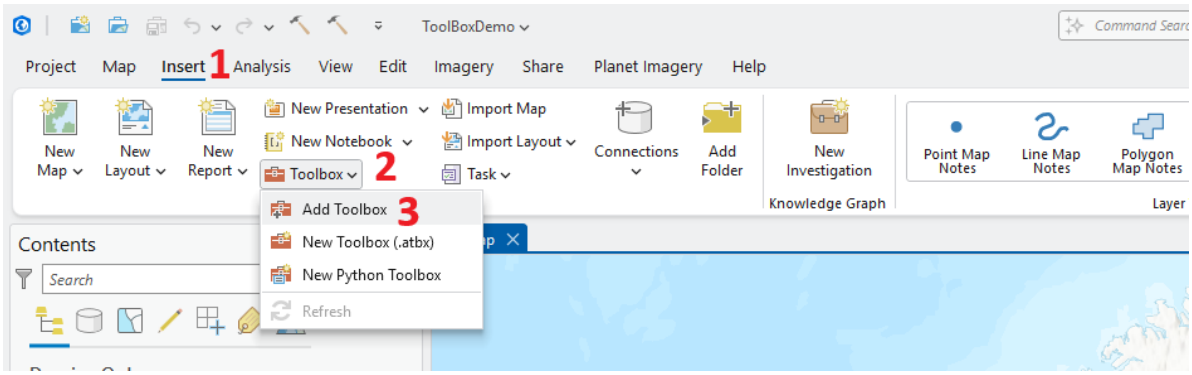


Figure 2: Adding an ArcGIS Python toolbox through the Catalog pane.

The main road-segmentation toolbox performs imagery export, reusable-cache handling, neural-network inference, probability reconstruction, and final output creation. The probability-threshold toolbox operates on an existing probability raster and does not load an ONNX model.

5.2 Main Road Segmentation Interface

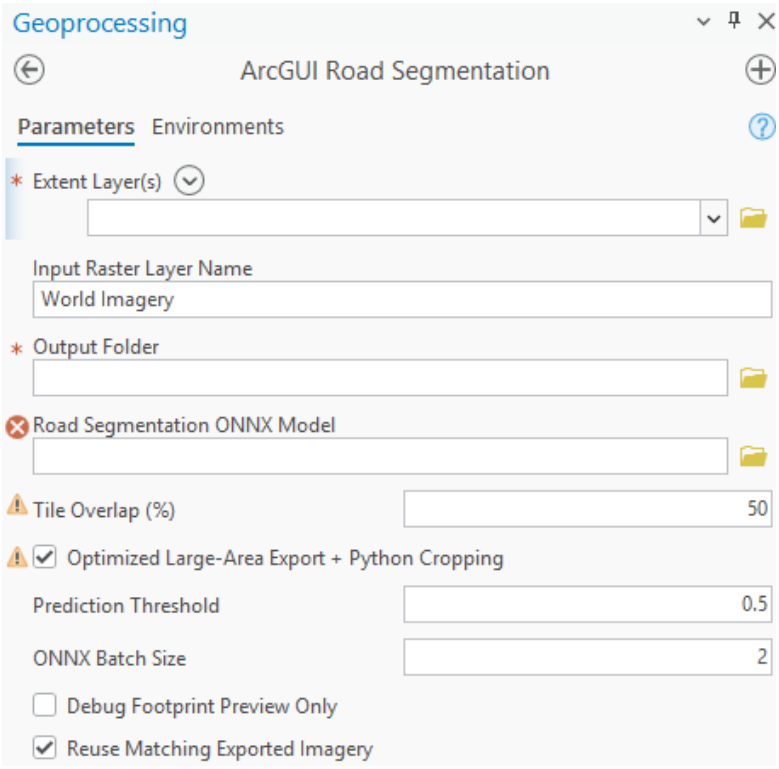


Figure 3: Image to add: Current *ArcGUI Road Segmentation* interface showing the ten final parameters: Extent Layer(s), Input Raster Layer Name, Output Folder, Road Segmentation ONNX Model, Tile Overlap, Optimized Large-Area Export, Prediction Threshold, ONNX Batch Size, Debug Footprint Preview Only, and Reuse Matching Exported Imagery.

For most users, a good first production run is: choose the polygon extent, keep `World Imagery`, select an output folder, choose `segformer_cuda_fp16.onnx`, keep 50% overlap, keep optimized mode enabled, keep threshold 0.50, keep the default batch size, and leave reusable imagery enabled.

The current GUI parameters are listed in Table 3. Yellow warning icons are normally informational and explain derived stride, model execution mode, optimized export behaviour, or reusable-cache status. Red error icons indicate a condition that must be corrected before the tool can run safely.

Table 3: Main ArcGUI parameters.

Parameter	**behaviour**
Extent Layer(s)	Required polygon feature-layer input. Multiple layer entries can be supplied. ArcGUI rejects duplicate entries, non-polygon inputs, empty geometry, and extents whose coordinate system is unknown. Current selections and definition queries are preserved in the processing copy.
Input Raster Layer Name	Name of one unique imagery/raster layer in the active map, normally `World Imagery`. The name must match exactly and the active map must use a projected coordinate system in metres.
Output Folder	Root folder for shared imagery caches, model-specific runs, final predictions, and debug products. ArcGUI verifies that the folder can be written and checks available disk space.
Road Segmentation ONNX Model	Select one supported SegFormer MiT-B5 or U-Net ResNet101 ONNX model. For normal production use, choose `segformer_cuda_fp16.onnx` or `best_unet_resnet101_norm_1024_cuda_fp16.onnx`. The model is not required in debug-only mode.
Tile Overlap (%)	Allowed range 0–95%. The default is 50%. ArcGUI converts this percentage to model-window spacing automatically. High overlap creates many more model windows and therefore increases processing time.
Optimized Large-Area Export + Python Cropping	Recommended for large extents. ArcGIS exports balanced large rectangular TIFF blocks and Python creates the overlapping model windows locally. When unchecked, ArcGIS directly exports the retained 1024×1024 model tiles.
Prediction Threshold	Allowed range 0–1. The default is 0.50. Thresholding is performed after overlapping probabilities have been averaged.
ONNX Batch Size	Number of model windows submitted per ONNX Runtime call. The default changes with the selected model profile. Keep the automatic/default value unless the workstation has been tested with a different batch size.

Parameter	behaviour
Debug Footprint Preview Only	Runs the spatial planner without imagery export and without ONNX inference. Use it to verify the selected extent, optimized blocks, and model-window coverage before a long job.
Reuse Matching Exported Imagery	Recommended to leave enabled. If the same extent and export configuration were already processed, ArcGUI can reuse the existing imagery TIFFs and run a different model without downloading the same imagery again.

5.3 Recommended Starting Batch Sizes

Table 4: Conservative starting batch sizes.

Deployment	Starting batch size
SegFormer CUDA	2
SegFormer CPU	1
U-Net ResNet101 CUDA	4
U-Net ResNet101 automatic/fallback	4

If GPU memory is insufficient, reduce the ONNX batch size before changing spatial parameters. The model input size should remain fixed at 1024×1024 .

6 Coordinate Systems, Extents, and Geometry Handling

6.1 ArcGIS Pro Environment Settings

ArcGIS Pro also provides an *Environments* tab in the Geoprocessing pane for controlling spatial processing settings. Figure 4 highlights two settings that are particularly useful when preparing a road-segmentation job.

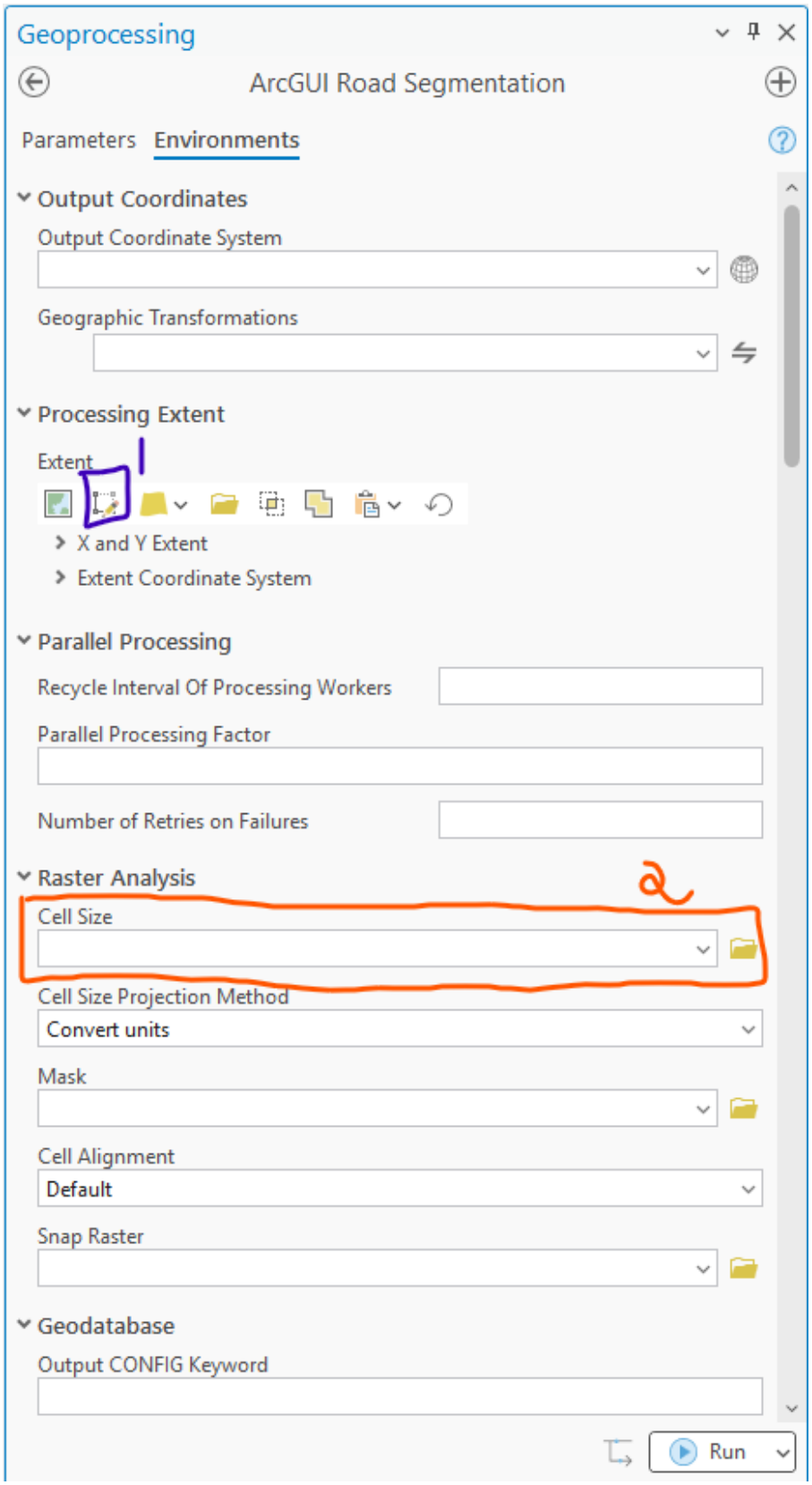


Figure 4: ArcGIS Pro *Environments* tab showing the Processing Extent control (1) and the Cell Size setting (2).

The control identified by **1** is located under *Processing Extent*. The rectangular extent-drawing tool can be used to interactively define a rectangular area of interest directly from the map. This can be convenient when a simple rectangular study area is required and an appropriate polygon extent has not already been prepared. For the main ArcGUI workflow, the polygon supplied through *Extent Layer(s)* remains the authoritative processing area. If a custom rectangular area is drawn in the ArcGIS environment settings, it should therefore be converted or otherwise represented by the polygon extent supplied to ArcGUI when that rectangle is intended to define the actual road-extraction area.

The control identified by **2** is the *Cell Size* setting under *Raster Analysis*. The ArcGUI deployment was designed for imagery processed at 2 map units per pixel. Because ArcGUI requires a projected coordinate system whose map units are metres, a cell size of 2 corresponds to approximately **2 metres per pixel**. When manually configuring ArcGIS geoprocessing environments for related raster operations, the Cell Size should therefore be set to 2 m to remain consistent with the spatial resolution used by the road-segmentation workflow.

The other environment settings shown in Figure 4 normally do not need to be modified for standard ArcGUI processing unless a specific GIS workflow requires them.

6.2 Map / Export Coordinate System

ArcGUI uses the active ArcGIS Pro map coordinate system as the normal spatial reference for imagery export and processing. The active map must have a valid projected coordinate system whose units are metres because the deployment resolution is fixed at 2.0 m/pixel. A geographic map in degrees or a projected map in feet is rejected before a production run begins. The requested polygon is projected into the map/export coordinate system before export blocks or model windows are planned. This keeps the coordinates used for planning consistent with the coordinate system written to the exported TIFF files.

6.3 Extent Layers with a Defined CRS

The extent layer must have a defined coordinate system. When its CRS differs from the active map CRS, ArcGUI projects a temporary processing copy into the map CRS. The original user dataset is not modified.

6.4 Extent Layers with an Unknown CRS

The current hardened build does **not** guess an unknown extent coordinate system. If the extent layer has an Unknown or undefined CRS, ArcGUI stops and asks the user to define the correct coordinate system before processing. This behaviour is intentional because guessing a CRS can place imagery and predictions in the wrong geographic location while still producing apparently valid files.

6.5 Polygon Validation, Selections, and Definition Queries

Only polygon geometry is accepted as an extent input. For every extent-layer entry, ArcGUI creates a temporary processing copy so the current feature selection and definition query are preserved. The temporary geometry is repaired where possible and null polygons are removed. If no usable polygon remains, the job stops. If ArcGIS cannot read the requested geometry or evaluate an intersection safely, ArcGUI stops rather than assuming that everything intersects.

6.6 Single Layer, Multiple Polygons, and Multiple Layer Entries

One entry in the multivalue *Extent Layer(s)* control is one processing job. If one layer contains several selected polygons, those polygons are processed together as the geometry for that job. If separate final products are required for individual polygons, provide them as separate layer entries or prepare separate layers/selections. ArcGUI rejects the same extent layer being listed more than once in the same run. When several different extent entries are supplied, ArcGUI processes them sequentially and reports success or failure for each one. If all requested extents fail, the toolbox raises an actual execution failure.

7 Fixed Model Windows and Overlap

Every inference window remains 1024×1024 . The user controls only the spacing between neighboring window origins.

For overlap percentage p:

$$s = \text{round}\left(1024\left(1 - \frac{p}{100}\right)\right).$$

Table 5: Examples of overlap and resulting stride.

Overlap	Model window	Stride
0%	1024×1024	1024 px
25%	1024×1024	768 px
50%	1024×1024	512 px
75%	1024×1024	256 px
95%	1024×1024	51 px

The final window origin on each axis is shifted when necessary so that the far edge is covered instead of leaving an uncovered strip. At 50% overlap, a typical interior pixel can be seen by multiple model windows. The overlapping probabilities are averaged, so the dense coverage is intentional and is not equivalent to a single 512-pixel model tile.

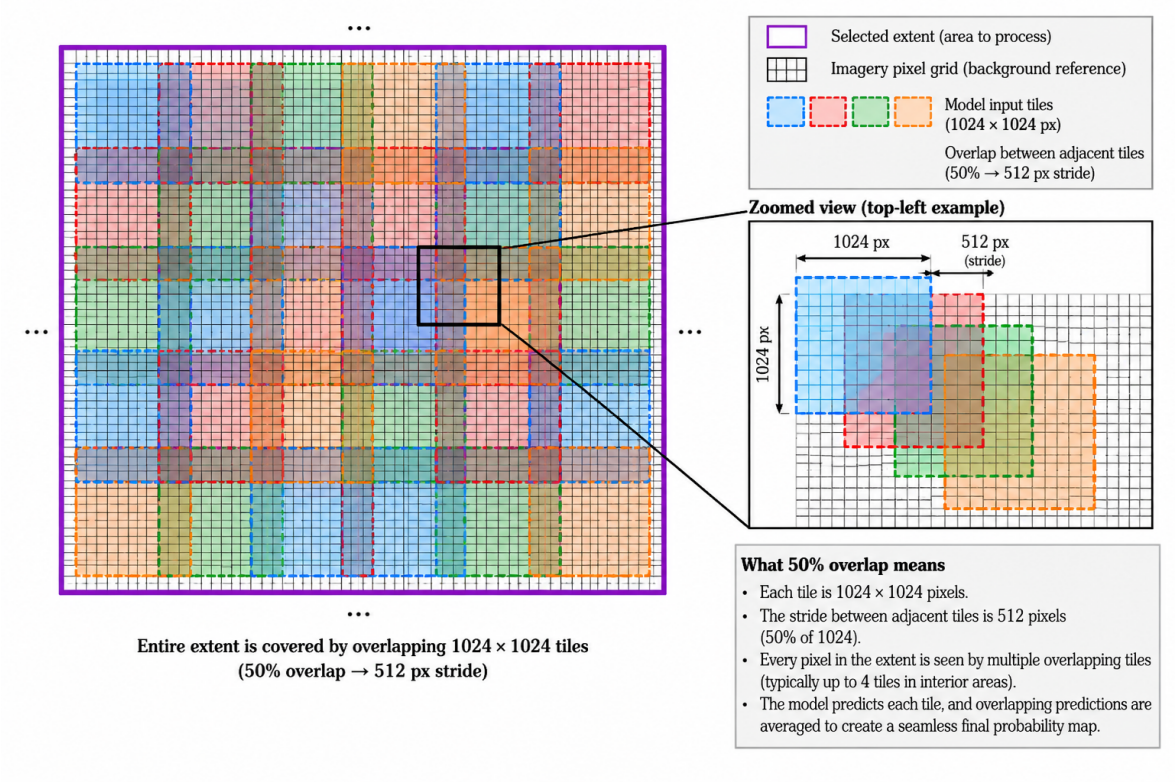


Figure 5: Direct sliding-window coverage. Every model input remains 1024 × 1024 pixels; 50% overlap means neighboring origins are separated by 512 pixels and interior pixels are predicted multiple times.

8 Processing Mode 1: Direct ArcGIS 1024-Tile Export

When *Optimized Large-Area Export + Python Cropping* is unchecked, ArcGUI uses direct model-sized imagery exports. The model tile size remains 1024 × 1024 and the geographic origin spacing is derived from the selected overlap percentage. Before imagery export, ArcGUI builds the complete direct-tile plan and removes candidate windows that have zero intersection with the exact selected polygon. Boundary-intersecting windows are retained in full so the model receives normal image context around the target edge.

The direct workflow is:

1. Validate the selected polygon, active map CRS, imagery layer, overlap, output folder, and model settings.
2. Build the exact 1024 × 1024 sliding-window plan using the requested overlap.
3. Skip candidate windows with zero intersection with the selected polygon before imagery export.
4. Export only the retained direct imagery windows.
5. Validate exported TIFF georeferencing and imagery content.

6. Ignore a completely black duplicate only when a valid TIFF covers the same required footprint.
7. Preprocess RGB imagery and run ONNX inference.
8. Save georeferenced probability tiles.
9. Stop instead of creating a partial mosaic if required target coverage is missing or invalid.
10. Average overlapping probabilities into `roads_probability_mosaic.tif`.

Direct mode is simple and useful for small or moderate extents.

For large or irregular regions, optimized mode is normally faster because it reduces the number of ArcGIS imagery-export operations.

9 Processing Mode 2: Optimized Large-Area Export

Optimized mode separates ArcGIS imagery extraction from neural-network model tiling. ArcGIS exports one or more large non-overlapping rectangular TIFF blocks. Python then creates the overlapping 1024×1024 model windows locally inside each retained large TIFF.

9.1 Default Large-Block Limit

The default maximum large export is 10240×10240 pixels. This corresponds to ten model-window widths along each axis. The default is a planning cap rather than an unconditional promise that an external image service will accept a request of that size. If ArcGIS exposes a smaller `maxImageWidth` or `maxImageHeight`, ArcGUI reduces the starting block cap automatically.

9.2 Balanced Non-Overlapping Partitioning

For working width W and height H, the minimum default numbers of large-block columns and rows are:

$$N_x = \left\lceil \frac{W}{10240} \right\rceil, \qquad N_y = \left\lceil \frac{H}{10240} \right\rceil.$$

Each axis is divided as evenly as possible.

For an axis length L split into N sections:

$$q = \left\lfloor \frac{L}{N} \right\rfloor, \qquad r = L \bmod N.$$

The first r sections receive $q + 1$ pixels and the remaining sections receive q pixels.

Neighboring block dimensions therefore differ by at most one pixel.

Table 6: Representative default optimized planning cases before service-limit fallback.

Working size	Grid	Approx. block size	Candidates
7000×7000	1×1	7000×7000	1
12000×12000	2×2	6000×6000	4
18000×7000	2×1	9000×7000	2
20000×20000	2×2	10000×10000	4
25000×12000	3×2	8333×6000	6

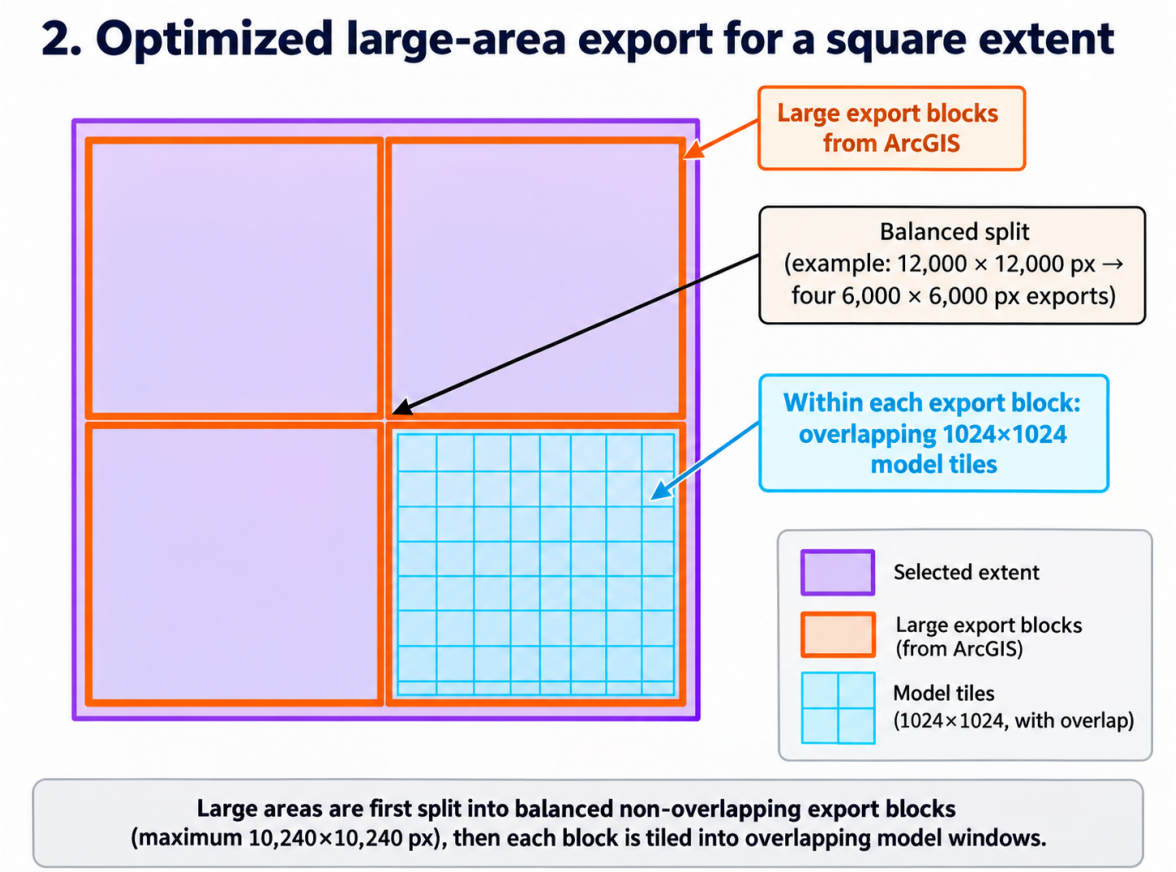


Figure 6: Optimized large-area processing for a regular extent. Large ArcGIS export blocks are balanced and non-overlapping, while overlapping model windows are created later inside each retained TIFF.

9.3 Skip Large Blocks That Do Not Intersect the Target

The balanced grid is first planned over the working bounding rectangle. Every candidate large block is then tested against the actual exact selected polygon geometry. A candidate

with zero intersection is skipped **before** World Imagery export. Therefore an orange planning block that lies completely outside an irregular target no longer consumes an imagery-service request, local disk space, or model inference time. A retained large block is still exported as a rectangle even if only part of it intersects the target. Inside that retained TIFF, Python skips model windows that are completely outside the target geometry.

9.4 Irregular Polygon Extents

There is no separate irregular-mode checkbox. ArcGUI always plans rectangular imagery blocks, but the polygon remains the authoritative processing geometry. For an irregular polygon, the bounding rectangle supplies the regular planning frame while the polygon is used to determine which large blocks and model windows are actually relevant. The workflow does not depend on outside-polygon imagery being black. The final products are clipped back to the requested polygon.

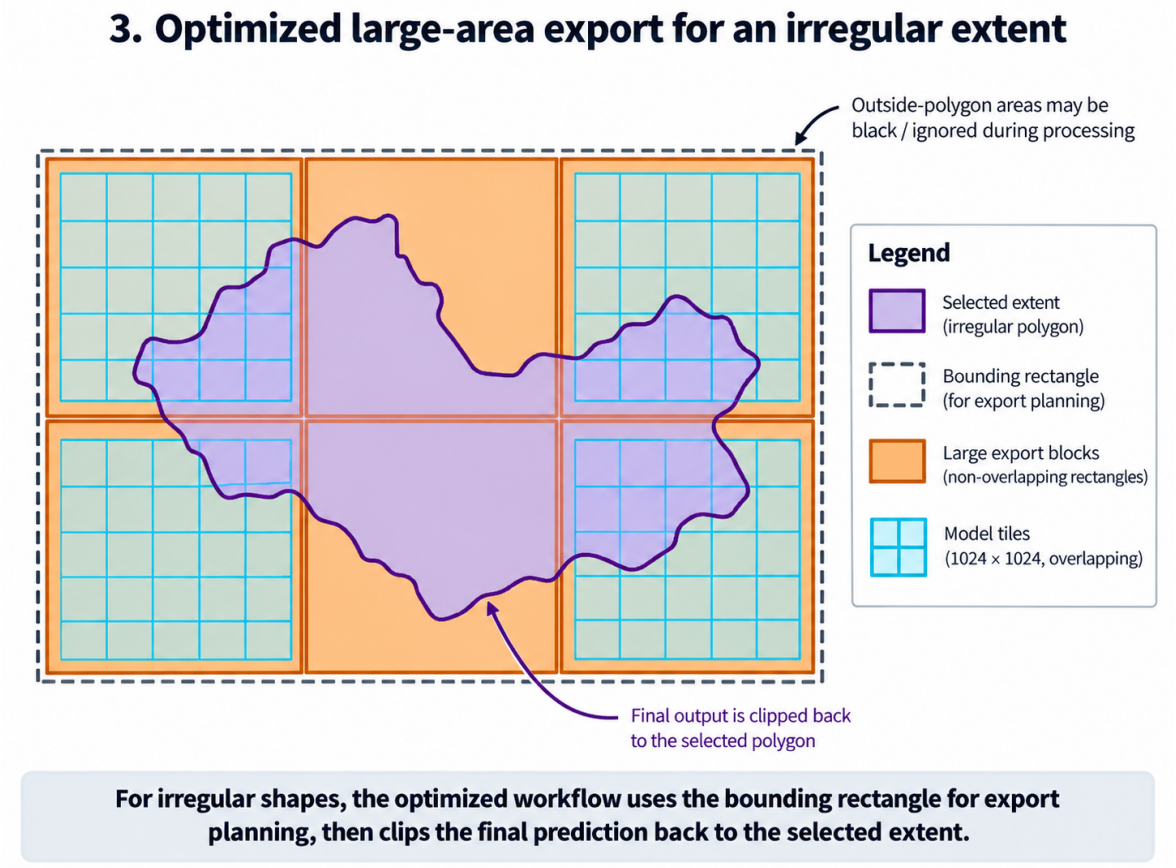


Figure 7: Optimized large-area processing for an irregular polygon. The bounding rectangle supports regular block planning, non-intersecting blocks can be skipped before export, and the polygon remains the final target geometry.

9.5 Image-Service Request Limits and Automatic Smaller-Block Retry

A 10240×10240 client request can still exceed an imagery service's own request limit. ArcGUI therefore attempts to read image-service maximum request dimensions when ArcGIS exposes them. If the service reports a safe maximum below 10240 pixels, the optimized planner uses the smaller value from the start. If a large request fails with a likely request-size error, ArcGUI replans the *entire* optimized export using progressively smaller balanced, non-overlapping blocks.

The fallback sequence is approximately:

```
10240 -> 5120 -> 2560 -> 1280 -> 1024 pixels
```

A generic ArcGIS imagery-export failure receives one smaller-block recovery attempt even when the service does not clearly report a size-limit message. A true imagery-content error or georeference error is not treated as a request-size problem and is not hidden by repeated retries. If the service limit is below 1024 pixels, ArcGUI stops because the fixed model chip itself cannot be requested safely.

9.6 Multiple TIFFs from One Large Export

ArcGIS can sometimes create more than one TIFF in a large-block export folder. One TIFF can be the correct World Imagery result while another can be a completely black duplicate or edge artifact. ArcGUI inspects every candidate. Completely black TIFFs are ignored. Unreadable TIFFs are not selected. Among usable non-black TIFFs, ArcGUI selects the candidate whose dimensions best match the planned block. If ArcGIS returns only partial non-black TIFFs that do not cover the planned block dimensions closely enough, ArcGUI treats the event as a likely request-size problem and can retry with smaller large blocks. If every TIFF for a retained target-intersecting block is black or unreadable, ArcGUI raises an imagery-content error instead of silently producing a probability map with missing coverage.

9.7 Large-Block Boundary Limitation

The large ArcGIS blocks themselves intentionally do not overlap. Model windows overlap inside each individual large block but do not cross a large-block boundary. This keeps large-area extraction simple and avoids redundant large imagery downloads. A visible seam can therefore theoretically occur at a large-block boundary because windows on one side do not receive imagery context from the neighboring block. Increasing model-window overlap does not make windows cross that boundary.

10 Reusing Previously Exported Imagery

Large-area imagery export can take longer than model inference, so the current ArcGUI stores exported imagery in a reusable configuration-aware cache. This is particularly useful when comparing SegFormer and U-Net on the same geographic area.

For example, a user can process one extent with `segformer_cuda_fp16.onnx` and later run `best_unet_resnet101_norm_1024_cuda_fp16.onnx` without downloading the same imagery again when the export configuration still matches.

The *Reuse Matching Exported Imagery* option is enabled by default. A cache match depends on the selected extent geometry, extent/map coordinate systems, imagery source, direct versus optimized mode, overlap percentage, fixed tile size, fixed 2 m/pixel resolution, and the current cache schema. The selected ONNX model, prediction threshold, and ONNX batch size do **not** change the imagery cache identity.

When a matching cache is found, the GUI displays a yellow cache-hit warning and production validates the cached TIFFs again before ONNX inference. If the cache is incomplete, corrupted, spatially inconsistent, or does not match the current plan, ArcGUI rejects it and creates a fresh imagery export rather than silently reusing questionable files. Older pre-r7 cache layouts are intentionally not reused automatically because the current release uses the exact-selected-polygon policy and a newer cache schema.

A small text and JSON manifest are stored with each cache so the export configuration can be reviewed later. This makes model comparison more efficient while keeping the imagery export separate from model-specific prediction outputs.

11 Debug Footprint Preview

The *Debug Footprint Preview Only* mode is intended to inspect spatial planning before a long imagery export. Debug mode does not export World Imagery, does not load the ONNX model, and does not run inference. The active map CRS is used for planning so debug and production use the same spatial frame.

If the named imagery layer is present, debug mode can also inspect image-service request limits and represent the same optimized block cap production is expected to use. The current GUI validates the named imagery layer and the active projected metric map before debug or production planning, so the preview reflects the same spatial setup that will be used for the real run.

For irregular or projected polygons, ArcGUI validates the projected extent and reconstructs the bounding rectangle from actual polygon geometry if ArcGIS returns NaN extent metadata. Every debug run receives a new debug folder and file geodatabase so old preview layers do not cause schema-lock failures.

In optimized mode the preview distinguishes:

- large blocks that will be exported;
- candidate large blocks skipped because they do not intersect the requested geometry;
- active 1024×1024 model windows;
- model windows skipped because they lie completely outside the requested geometry.

At 50% overlap, the visible model-window origin spacing is 512 pixels even though every footprint remains 1024 pixels wide and high. To avoid accidentally freezing ArcGIS Pro, debug mode refuses to create a preview that would require more than 500,000 model-crop polygon features. If all requested debug previews fail, the toolbox raises an error instead of ending with only a warning.

12 Georeference and Raster-Alignment Validation

The hardened build validates exported imagery before inference. The checks are designed to catch major coordinate-system mistakes while allowing small raster-alignment differences introduced by ArcGIS.

Optimized large TIFF validation includes:

- a usable spatial reference;
- compatibility with the chosen map/export CRS;
- finite extent coordinates;
- finite positive cell size;
- cell size close to the intended 2 m/pixel deployment resolution;
- raster dimensions close to the planned block dimensions;
- planned and actual geographic extent compatibility;
- raster-center consistency.

These checks prevent the workflow from continuing with a raster whose coordinate values belong to one projection while its TIFF metadata claims another projection. Probability rasters are also checked for compatible CRS and cell size before they are mosaicked. The first probability raster is used as the snap raster for mosaicking. Final clipping uses the original probability grid alignment rather than forcing a new grid merely to match the exact polygon envelope.

13 Probability Products, Thresholding, and Final Outputs

13.1 Probability Mosaic

The principal reusable inference product is:

`roads_probability_mosaic.tif`

This raster stores the reconstructed continuous road probabilities before the user-selected threshold is applied.

The probability mosaic should be retained whenever thresholds may need to be explored later.

13.2 Finalization Products

For each successful extent, the finalization stage creates:

`roads_probability_mosaic.tif`
`roads_probability_clipped_to_extent.tif`
`roads_prediction_binary.tif`
`<extent>_<model>_roads_prediction_clean.tif`

The **probability mosaic** is the reconstructed continuous surface.

The **clipped probability raster** is the probability surface restricted to the requested target geometry.

The binary raster stores the immediate threshold result produced during the main run using 0 for background and 255 for road.

The clean road raster is the map-oriented version in which background is converted to NoData when possible, and it is copied into the shared `final_predictions` folder using a non-conflicting name.

If optional background-cleanup or display-maintenance operations fail, ArcGUI preserves the already valid binary prediction instead of discarding the entire successful segmentation run. If the selected threshold produces zero road pixels, ArcGUI issues a warning but does not treat that as a processing failure because a genuinely empty binary result can be valid. If the clipped probability raster contains only NoData, ArcGUI stops because that generally indicates missing imagery, a geometry mismatch, or another spatial failure.

13.3 Representative Output Structure

The current release separates shared imagery exports from model-specific prediction runs.

A representative optimized run is:

```
output_folder/
|-- extent_name/
|   |-- export_cache/
|   |   `-- optimized_ov50_<cache_id>/
|   |       |-- export_config.txt
|   |       |-- export_config.json
|   |       `-- imagery/
|   |           `-- large_exports_max_10240px/
|   |               |-- block_0001/
|   |               `-- ...
|   `-- model_outputs/
|       |-- segformer_best_cuda/
|       |   `-- optimized_ov50_<cache_id>/
|       |       `-- run/
|       |           |-- model_run_info.txt
|       |           |-- run_status.txt
|       |           |-- large_block_probabilities/
|       |           `-- final/
|       `-- unet_best_cuda/
|           `-- optimized_ov50_<same_cache_id>/
|               `-- run/
`-- final_predictions/
    |-- segformer_best_cuda/
    |   `-- optimized_ov50_<cache_id>/
    `-- unet_best_cuda/
        `-- optimized_ov50_<same_cache_id>/
```

The `export_cache` folder is shared across models when the export configuration matches. The `model_outputs` folder keeps each model run separate, so comparing SegFormer and U-Net does not overwrite earlier probability products or final results. Repeated runs of the same model/configuration create non-conflicting run or filename variants rather than deleting previous outputs that may still be in use by ArcGIS.

The most important finalization products inside each successful run are:

```
roads_probability_mosaic.tif
roads_probability_clipped_to_extent.tif
roads_prediction_binary.tif
<extent>_<model>_roads_prediction_clean.tif
```

Debug products are stored separately in non-conflicting debug-preview workspaces.

14 Probability Map Threshold Toolbox

The secondary `RoadProbabilityThreshold.pyt` toolbox is intended for post-processing an already generated probability map.

It does not export imagery and does not run ONNX inference.

14.1 Tool 1: Load / Refresh Threshold Preview

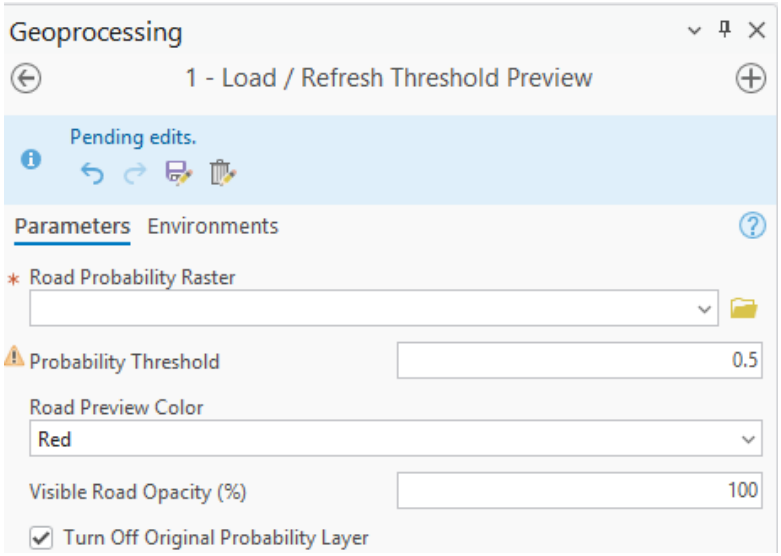


Figure 8: Load / Refresh Threshold Preview interface.

The inputs include the probability raster, threshold, preview colour, visible-road opacity, and an option to turn off the original continuous probability layer.

The preview changes map symbology only.

For threshold T:

$$P \leq T \rightarrow \text{transparent}, \qquad P > T \rightarrow \text{selected road colour}.$$

No new full threshold raster is written by the preview tool.

Changing the threshold and running the preview again refreshes the same conceptual display without repeating neural-network inference. A standard ArcGIS Python toolbox does not provide a true continuously dragged numeric slider that can safely run arbitrary map processing during every mouse movement.

14.2 Tool 2: Generate Binary Road Mask

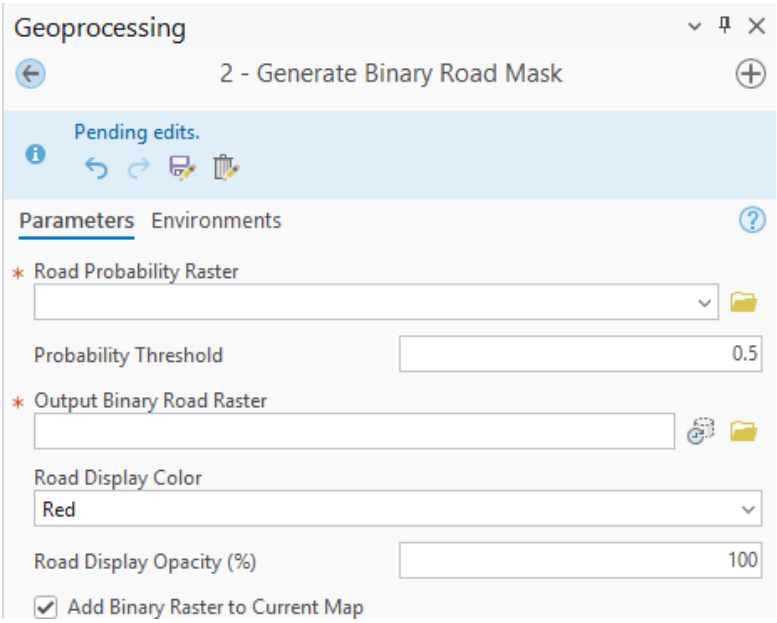


Figure 9: Generate Binary Road Mask interface.

This tool creates a permanent binary raster using:

$$P > T \rightarrow 1, \qquad P \leq T \rightarrow 0.$$

Source NoData is preserved as NoData.

The selected display colour affects map visualization only and does not change the stored road value of one.

14.3 Native ArcGIS Symbology as a Slider-Like Alternative

A probability raster can also be explored without the threshold toolbox. Add `roads_probability_mosaic.` to the map and open its raster Symbology. Use a two-class *Classify* renderer with a manual class break at the desired threshold. Set the lower class to fully transparent and the upper class to red or another road colour. The class break can then be moved numerically or adjusted on the raster histogram. This is the closest native ArcGIS equivalent to a visual threshold slider because the display updates without modifying the underlying probability data.

14.4 Recommended Threshold-Selection Workflow

A practical workflow is:

1. Run the main segmentation toolbox once.

2. Retain `roads_probability_mosaic.tif`.
3. Inspect candidate thresholds with either the threshold-preview toolbox or native ArcGIS classify symbology.
4. Compare operational thresholds such as 0.30, 0.40, 0.50, 0.60, and 0.70 when useful.
5. Prefer a validation-selected operating threshold when quantitative validation is available.
6. Use visual inspection to understand the spatial consequences of that threshold.
7. Run *Generate Binary Road Mask* to create the permanent binary product.
8. Retain both the probability raster and the final binary raster when reproducibility is important.

15 Runtime Safeguards and Failure behaviour

The hardened version is intentionally conservative when continuing could create a spatially incomplete, incorrectly georeferenced, or misleading result.

15.1 Output Directory and Disk Space

ArcGUI creates the output root if required and performs a write test before processing. Available disk space is checked because large TIFF exports and probability rasters can consume substantial temporary storage. Very low free space can produce a warning or hard stop before a long job begins.

15.2 Reusable Cache Validation

A cache hit is treated as a candidate for reuse rather than blindly trusted. Before ONNX inference, ArcGUI verifies that the cached files still exist, match the expected processing plan, and remain compatible with the current georeferencing and imagery configuration. If validation fails, the questionable cache is rejected and a fresh imagery export is created.

15.3 Black or Missing Imagery

Completely black duplicate TIFFs are ignored when a valid duplicate exists. A retained target-intersecting large block for which every candidate TIFF is black causes a hard imagery-content error. Direct mode also stops when black imagery is the only coverage for a target-intersecting footprint. This behaviour is intentional because silently skipping required imagery would create a road map with an undocumented spatial hole.

15.4 Inference Failures

Direct mode refuses to create a partial probability mosaic when one or more required direct tiles cannot be prepared, inferred, or saved. Optimized mode stops if no valid block-level probability raster can be produced. NaN or infinite network outputs are rejected. Batch inference can fall back to single-image inference for an individual batch when appropriate, but the run still fails if required imagery cannot ultimately produce valid probabilities.

15.5 All Extents or All Debug Jobs Failing

Multi-extent jobs record failures per extent and continue to later entries where possible. If at least one extent succeeds, successful outputs are retained and failed entries are reported in the geoprocessing messages. If all production extents fail, the toolbox raises an execution error. If all debug previews fail, debug mode also raises an execution error.

16 Troubleshooting

16.1 CUDA Model Selected but CUDA Is Not Available

If `segformer_cuda_fp16.onnx` is selected and `CUDAExecutionProvider` is unavailable, ArcGUI stops intentionally.

Verify the active cloned environment, NVIDIA driver, ONNX Runtime GPU package, CUDA/cuDNN compatibility, and provider list.

Use `segformer_cpu_fp32_optimized.onnx` when a CPU-only path is required.

16.2 CPU Inference Is Very Slow

SegFormer MiT-B5 is a large model and CPU inference is expected to be substantially slower than CUDA inference.

Use the CPU-optimized graph and start with batch size 1.

16.3 GPU Out of Memory

Reduce the ONNX batch size.

Do not change the fixed 1024×1024 model input as a memory workaround.

16.4 Model Filename Is Rejected

Use one of the recognized SegFormer or U-Net ResNet101 naming families listed in Section 3.1.

For normal deployment, use `segformer_cuda_fp16.onnx` or `best_unet_resnet101_norm_1024_cuda_fp1`

ArcGUI identifies the architecture and execution target from the filename and then validates the actual ONNX input/output contract when the model is loaded.

16.5 Input Raster Layer Cannot Be Found

Production requires the imagery layer name entered in the GUI to match a layer in the active ArcGIS Pro map.

If several map layers have the same requested name, ArcGUI stops instead of guessing which imagery source to use; rename the intended imagery layer so its name is unique.

16.6 The Map Uses Degrees or Feet

Change the active map to a projected coordinate system with metre units.

The deployment cell size is defined in metres.

ArcGUI intentionally rejects geographic-degree maps and projected non-metric units.

16.7 Extent Layer Has an Unknown CRS

The current hardened release rejects an extent whose coordinate system is Unknown or undefined.

Define the correct source CRS on the extent dataset before processing, then verify that the polygon appears in the expected geographic location.

16.8 Debug Preview Reports NaN Extent Values

The current build reconstructs invalid projected extent metadata from the actual polygon geometries.

If the fallback also fails, repair the source geometry and verify that the active map CRS is valid and metric.

16.9 Optimized Large Export Is Rejected by the Image Service

ArcGUI first checks exposed image-service request limits where possible.

If a large request is still rejected, it retries with smaller balanced block caps down to 1024 pixels.

If requests continue to fail at the minimum block size, review connectivity, service availability, authentication, and ArcGIS geoprocessing messages.

16.10 A Large-Block Folder Contains a Correct TIFF and a Black TIFF

This behaviour can occur with *Export Training Data for Deep Learning*.

ArcGUI inspects both files and ignores the completely black duplicate when a usable non-black TIFF matches the planned block.

The black file can remain on disk for troubleshooting but is not passed to the model.

16.11 Every TIFF for a Retained Block Is Black

The run stops because the block intersects the requested geometry and therefore represents required imagery coverage.

Check World Imagery availability, extent location, coordinate system, map projection, service connectivity, and export messages.

16.12 Large Blocks Appear in the Wrong Location

Confirm that the active map is using a projected metric CRS and review the logged source and map/export CRS values.

ArcGUI projects the requested extent before planning blocks and validates the georeferencing of exported TIFFs.

A major planned-versus-actual coordinate mismatch now produces an error instead of continuing to inference.

16.13 Irregular Polygon Produces Rectangular Temporary Imagery

This is expected.

Optimized imagery extraction uses rectangular blocks, while the polygon controls block/crop relevance and final clipping.

Temporary imagery outside the polygon does not imply that the final road result will extend outside the requested geometry.

16.14 Prediction Artifacts Near Model-Tile Edges

Increase the overlap percentage if additional model-window context averaging is desired.

More overlap causes each interior pixel to be predicted by more windows, after which the probabilities are averaged.

16.15 Prediction Artifacts at Large-Block Boundaries

Model windows do not cross optimized large-block boundaries.

If a visible seam remains important operationally, direct mode can be used for a comparison because it does not introduce the same large-block partition structure.

16.16 Debug Preview Is Too Dense

High overlap over a very large extent can produce hundreds of thousands of footprint polygons.

The current build stops before creating more than 500,000 model-crop preview features.

Reduce the preview extent, reduce overlap for planning inspection, or inspect the region in several smaller extents.

16.17 A Cache Exists but ArcGUI Exports Imagery Again

This normally means the current export configuration does not exactly match the stored cache configuration, the cache is incomplete, or the cache was created by an older incompatible release.

Check the selected extent geometry, active map CRS, imagery source, overlap percentage, and direct/optimized mode.

ArcGUI intentionally prefers a new export over reusing a cache that cannot be validated safely.

16.18 The GUI Shows a CACHE HIT Warning

This is expected and desirable when the same area and export settings were processed previously.

ArcGUI will validate the cached TIFFs again during execution, skip the imagery download when the cache remains valid, and run the currently selected model on the existing imagery.

16.19 Threshold Preview Does Not Update Continuously While Dragging

The custom threshold preview is a standard Python geoprocessing tool, not an ArcGIS Pro SDK dockable control.

Change the threshold and click Run to refresh the preview.

For more slider-like visual exploration, use the native raster Classify histogram break.

16.20 All Extents Failed but ArcGIS Previously Said "Succeeded"

The current hardened toolbox raises an execution error when all requested extents fail.

A mixed multi-extent run can still finish with successful outputs and warnings for the failed

entries.

17 Recommended Operational Workflow

For a new area, the following sequence is recommended:

1. Open the intended imagery layer in ArcGIS Pro, normally `World Imagery`.
2. Set the active map to an appropriate projected coordinate system in metres.
3. Verify that the extent polygon is geographically correct and has a defined CRS.
4. Apply any required feature selection or definition query.
5. Select an output folder with sufficient free disk space.
6. For normal production, choose `segformer_cuda_fp16.onnx`; choose `best_unet_resnet101_norm_10` when faster inference is preferred.
7. Keep 50% overlap as a practical starting value unless validation or visual inspection supports another setting.
8. Keep *Optimized Large-Area Export + Python Cropping* enabled for large regions.
9. Keep *Reuse Matching Exported Imagery* enabled so later model comparisons can reuse the same exported imagery when the configuration matches.
10. Optionally run *Debug Footprint Preview Only* first to inspect retained/skipped blocks and model-window coverage without downloading imagery or loading a model.
11. Run production and review any yellow warnings or red errors shown in the Geoprocessing pane.
12. Inspect the probability mosaic and the final binary/clean road output.
13. If needed, explore another threshold using the probability-threshold toolbox without repeating model inference.
14. To compare the other recommended model on the same area, keep the extent, overlap, imagery source, and processing mode unchanged, select the other ONNX model, and run again; a valid matching cache should avoid another imagery export.
15. Retain the probability raster, model filename, threshold, cache configuration, and final road raster when reproducibility is important.

18 Deployment Summary

ArcGUI Road Segmentation provides a complete ArcGIS Pro workflow for automated forestry and agricultural road segmentation using fixed 1024×1024 RGB model windows.

The two recommended production models are `segformer_cuda_fp16.onnx` and `best_unet_resnet101_nor`

SegFormer is the primary final model, while the best U-Net CUDA model provides a faster alternative with broadly comparable road extraction results.

CPU, last-checkpoint, and old-milestone model files remain available mainly for compatibility, comparison, and reproducibility.

Direct mode exports retained 1024×1024 model tiles, while optimized mode exports balanced large TIFF blocks and creates overlapping model windows locally.

Both modes use the exact selected polygon as the target geometry and avoid unnecessary work outside that geometry where possible.

Overlapping model predictions are combined by mean probability before thresholding, and the continuous probability surface is retained for later inspection or re-thresholding.

The reusable imagery cache is a major practical optimization: changing the model, threshold, or batch size does not require another imagery export when the extent and export configuration still match.

The final hardened implementation validates coordinate systems, extent geometry, imagery source, export integrity, cache integrity, model contracts, CUDA/CPU provider behaviour, disk/output state, and per-extent completion so incorrect or partial results fail visibly rather than silently.

The included `check.py` utility provides a simple pre-deployment model/CUDA test, while `python_inference_script.py` provides a minimal non-ArcGIS folder-inference example for demonstration and sanity checking.

For routine operational use, the main ArcGIS toolbox remains the authoritative workflow because it preserves georeferencing, overlap reconstruction, exact polygon clipping, reusable imagery export, and model-specific output organization.